

GitHub草キーホルダー 設計まとめ

Sep 28, 2026 · @TatsuyaM

概要

直近365日のGitHubコントリビューション（草）を、半年ずつ2枚のキーホルダーにする。1枚は色ごとに単色で印刷した層を重ね、はめ込みで組み立てる。4色までのプリンタでも作れるようにするための構成。

1. GitHub GraphQL APIで直近1年分（約53週）を取得する
2. 前半と後半に分け、1枚あたり27週×7日のグリッドにする
3. Pythonスクリプト（`grass_keychain.py`）で、層ごとのSTLを生成する
4. 層ごとに単色で印刷し、上から 白→L1→L2→L3→L4 の順に重ねてはめ込む

検討の経緯

多色で一括印刷する案から始めたが、色数と強度の問題で、色ごとに層を分けてはめ込む積層式に変えた。

段階	案	結果
1	多色一括印刷（ベース+L0～L4の6パーツ、1枚の3MF）、マス1.47mm、外形62.5×16.7mm	後半のプレートに6色必要で、手元のプリンタ（4色まで）では印刷できない
2	マスを5mm角に拡大	外形が約158×41mmになり、キーホルダーには大きすぎる
3	マス3.0mm角を採用	外形103.9×27.4mm（マスの間隔0.4mmのとき）
4	色ごとの積層はめ込み式に変更	各層を単色で印刷できる。層の壁を確保するためマスの間隔を1.0mmに広げ、外形は119.5×31.0mm

決定した仕様

1枚の外形は119.5×31.0mmで、厚さは最大4.4mm（突起込み5.0mm）。ノズル0.4mmを前提にしている。

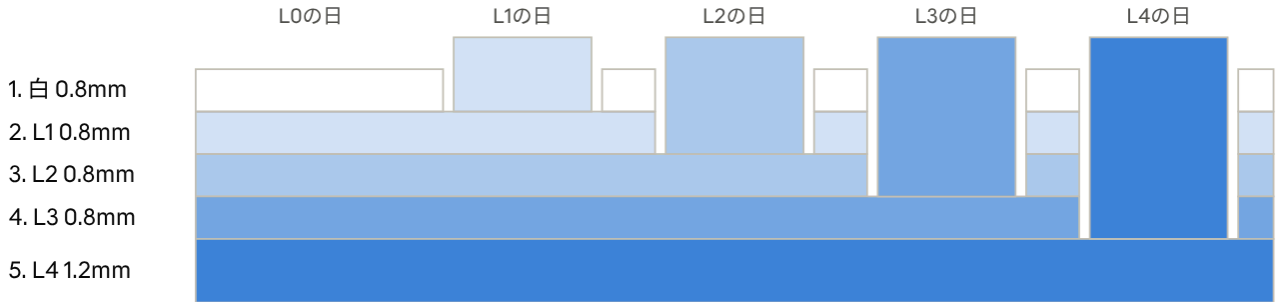
項目	値	オプション
枚数・期間	2枚（前半・後半の半年ずつ、1枚27週×7日、日曜が最上段）	—
マス	3.0mm角、角丸0.5mm	<code>--cell</code> / <code>--cell-r</code>
マスの間隔（くり抜き間の壁厚）	1.0mm	<code>--gap</code>
穴と突起のすき間	片側0.2mm（突起は2.6mm角）	<code>--clearance</code>
突起の飛び出し	白の上面から0.6mm	<code>--pin-protrude</code>
白・中間層の厚さ	0.8mm	<code>--layer-t</code>
最下層の厚さ	1.2mm	<code>--bottom-t</code>
キーリング穴	直径4.5mm、周りの肉厚2mm、左端のタブ、全層を貫通	<code>--hole-d</code> / <code>--hole-wall</code>
外形	マス目の周りに余白2mm、角丸3mm	<code>--margin</code> / <code>--corner-r</code>
裏面の刻印	最下層の裏のみ。ユーザー名、太字、高さ8mm、深さ0.6mm。長い名前は幅に合わせて縮小	<code>--back-text</code> / <code>--text-h</code> / <code>--text-depth</code>
L0（草のない日）	白のまま（くり抜かない）	—
固定方法	はめ込みのみ	—

積層構造

各層は、自分のレベルのマスが突起、自分より下の層のマスが穴になっている。下の層ほど突起が長く、上の層をすべて貫通する。

各層の突起は上の層の穴を貫通し、白の上面から0.6mm出る

断面図（plate2、5層。横は各レベルのマス1つずつ）



穴は3.0mm角、突起は2.6mm角（すき間は片側0.2mm）。L0の日は全層が埋まったままで、白が見える
L4の裏面にユーザー名を深さ0.6mmで刻印。各層は突起を上に向けて印刷する

積層構造の断面・5層

順番	層	厚さ	突起（層の上面からの高さ）	穴
1	白	0.8mm	なし	L1～L4のマス
2	L1	0.8mm	L1のマス（1.4mm）	L2～L4のマス
3	L2	0.8mm	L2のマス（2.2mm）	L3・L4のマス
4	L3	0.8mm	L3のマス（3.0mm）	L4のマス
5	L4	1.2mm	L4のマス（3.8mm）	なし（裏面に刻印）

マスが1つも無いレベルの層は作らない。そのため現在のデータでは、plate1（前半、L1のみ）は白+L1の2層で厚さ2.0mm、plate2は5層で厚さ4.4mmになる。最下層になった層が1.2mmになり、刻印が入る。

データ取得

GitHub GraphQL APIの `contributionsCollection.contributionCalendar` で直近1年分を取得する。REST APIには草のデータがない。各日の `contributionLevel`（`NONE`、`FIRST_QUARTILE`～`FOURTH_QUARTILE`）を、そのままL0～L4として使う。

- トークンは環境変数 `GITHUB_TOKEN` から読む。未設定なら `gh auth token` を使う
- 必要な権限：classicトークンなら `read:user`。fine-grainedトークンなら権限なしで公開分を取得できる
- 非公開リポジトリ分も含めるには、プロフィール設定の「Include private contributions on my profile」を有効にする
- ログイン中のGitHubアカウントは `gh auth status` または `gh api user --jq .login` で確認できる

- SSLの証明書エラー（`CERTIFICATE_VERIFY_FAILED`）が出たため、`truststore` で macOSキーチェーンの証明書を使うようにした

zshでのトークンの渡し方（画面に表示されず、履歴にも残らない）：

```
read -s "GITHUB_TOKEN?Token: " && export GITHUB_TOKEN
```

スクリプトの使い方

`grass_keychain.py` は、作業フォルダ内の `venv`（Python 3.11）で実行する。`venv`には `numpy`、`manifold3d`、`matplotlib`、`truststore`を入れてある。

```
venv/bin/python grass_keychain.py <GitHubユーザー名> # APIから
取得して生成
venv/bin/python grass_keychain.py <ユーザー名> --json out/<ユーザー名>
_<contributions.json> # 保存済みデータで再生成
venv/bin/python grass_keychain.py <ユーザー名> --demo # ランダムデータ
で試す
venv/bin/python grass_keychain.py -h # オプション
一覧
```

出力先は `out/`（`--out` で変更できる）。

ファイル	内容
<code><user>_plate{1,2}_{n}<層>.stl</code>	層ごとのSTL。印刷する向き（突起が上）。nは上から重なる順番
<code><user>_keychain.3mf</code>	組み立てた状態の確認用。層ごとに色付きパーツ
<code><user>_plate{1,2}_preview.png</code>	組み立て後の表面・裏面・断面と、各層の図
<code><user>_contributions.json</code>	取得した生データ。 <code>--json</code> で再利用できる

検証結果と印刷時の注意

実データで生成したモデルでは、層どうしの重なり体積が0で、すき間は設計どおり片側0.2mmだった。実際の印刷と、スライサーへの読み込みはまだ試していない。

- 全層の形状が正常な立体（多様体）として生成できた
- どの層の突起も、白の上面から0.6mm飛び出す高さでそろっている
- 裏面の刻印は裏から見て読める向き。長いユーザー名では自動で縮小される

印刷時の注意：

- 片側0.2mmは、プリンタによっては緩くて抜けやすい。本番の前に小さなテスト片ではまり具合を確かめる
- 突起の根元が1層目の広がり（エレファントフット）で太ると入らない。スライサーのエレファントフット補正を有効にする
- 白の層は壁1.0mmの格子状で薄い。印刷後にベッドから剥がすときはゆっくり扱う

未決事項と次のステップ

- ☐ 2枚の厚さをそろえるか決める（空の層も作ると、2枚とも5層、4.4mmになる）
- ☐ すき間のテスト片（0.1 / 0.15 / 0.2mm）を作って印刷し、すき間を決める
- ☐ 各層の厚さ（白・中間層0.8mm、最下層1.2mm）でよいか確認する。仮の値として採用している
- ☐ 層ごとのSTLをスライサーに読み込んで、形状を確認する
- ☐ 本番を印刷して組み立てる